

Basics and OS Role

This note covers core operating system concepts for placement preparation:

- Role of an operating system
 - Kernel space vs user space
 - System calls
 - Program vs process
 - Process states
 - Context switching
-

1. What Is an Operating System?

An operating system, or OS, is system software that acts as an interface between the user, application programs, and computer hardware.

Hardware by itself only provides raw resources such as CPU, memory, disk, keyboard, display, network card, and other devices. Applications cannot safely or directly control all these resources on their own. The operating system manages these resources and provides a controlled environment in which programs can run.

Examples of operating systems:

- Windows
- Linux
- macOS
- Android
- iOS

In simple terms:

The operating system manages hardware resources and provides services to programs and users.

2. Role of an Operating System

The OS has many responsibilities. For placements, you should understand the OS as a resource manager and a control program.

2.1 OS as a Resource Manager

The computer has limited resources:

- CPU time
- Main memory
- Disk space
- Input/output devices
- Network bandwidth
- Files

Many programs may want to use these resources at the same time. The OS decides:

- Which process gets the CPU
- How much memory each process receives
- How files are stored and accessed
- How devices are shared
- Which process is allowed to perform which operation

Example:

If you are running a browser, music player, code editor, and terminal at the same time, all of them need CPU and memory. The OS schedules them so that it looks like everything is running simultaneously.

2.2 OS as a Control Program

The OS controls program execution to prevent errors and misuse.

It ensures that:

- One program does not access another program's memory illegally
- User programs do not directly damage hardware
- Files are protected using permissions
- The CPU is not monopolized forever by one process
- I/O devices are accessed in a safe and organized way

Without the OS, every application would need to know how to directly control hardware, which would be unsafe, difficult, and inefficient.

3. Main Functions of an Operating System

3.1 Process Management

A process is a program in execution. The OS manages all running processes.

Process management includes:

- Creating and deleting processes
- Scheduling processes on the CPU
- Suspending and resuming processes
- Handling process synchronization
- Supporting inter-process communication
- Managing deadlocks

Example:

When you open a browser, the OS creates one or more processes for it. When you close the browser, the OS releases the resources used by those processes.

3.2 Memory Management

Main memory, or RAM, is limited. The OS manages memory allocation and deallocation.

Memory management includes:

- Keeping track of used and free memory
- Allocating memory to processes
- Deallocating memory when no longer needed
- Protecting one process's memory from another process
- Supporting virtual memory

Example:

If multiple programs are running, the OS ensures each process gets its own memory area. A calculator program should not be able to modify the memory of a banking application.

3.3 File Management

The OS manages files and directories on storage devices.

File management includes:

- Creating files
- Deleting files
- Reading files
- Writing files
- Renaming files
- Managing directories
- Maintaining file permissions

Example:

When you save a document, the application does not usually write directly to the disk hardware. It requests the OS to write data to a file.

3.4 Device Management

The OS manages input and output devices using device drivers.

Examples of devices:

- Keyboard
- Mouse
- Monitor
- Printer
- Disk
- USB drive
- Network card

Device management includes:

- Sending commands to devices
- Receiving data from devices
- Managing device queues

- Handling interrupts
- Providing a uniform interface to applications

Example:

An application does not need to know the internal details of every printer model. The OS and printer driver handle the device-specific work.

3.5 CPU Scheduling

The CPU can execute only one process at a time per core. If there are many ready processes, the OS decides which one should run next.

Common CPU scheduling goals:

- Maximize CPU utilization
- Minimize waiting time
- Minimize response time
- Improve throughput
- Ensure fairness

Common scheduling algorithms:

- First Come First Serve
- Shortest Job First
- Round Robin
- Priority Scheduling
- Multilevel Queue Scheduling

3.6 Security and Protection

The OS protects system resources from unauthorized access.

Security and protection include:

- User authentication
- File permissions
- Memory protection
- Process isolation
- Privileged instructions
- Access control

Example:

On Linux, a normal user cannot modify important system files unless they have administrator privileges.

4. Kernel

The kernel is the core part of the operating system.

It stays in memory while the system is running and directly manages hardware resources.

The kernel handles:

- Process scheduling
- Memory management
- System calls
- Device drivers
- Interrupt handling
- File system operations
- Inter-process communication

In simple terms:

The kernel is the central component of the OS that controls hardware and provides essential services to programs.

5. Kernel Space vs User Space

Modern operating systems divide memory and execution privileges into two major regions:

- User space
- Kernel space

This separation improves security, stability, and control.

5.1 User Space

User space is where normal application programs run.

Examples:

- Browser
- Text editor
- Games
- Media player
- Compiler
- Terminal commands

Programs in user space have limited privileges.

They cannot directly:

- Access hardware devices
- Modify kernel memory
- Execute privileged CPU instructions
- Access another process's private memory

If a user program needs a service such as file access or network communication, it must request the kernel through a system call.

5.2 Kernel Space

Kernel space is where the kernel runs.

Code running in kernel space has high privileges and can directly access:

- CPU control registers
- Physical memory
- Device controllers
- Kernel data structures
- Process tables
- File system internals

Because kernel space has full control over the system, bugs in kernel code can crash the entire OS.

5.3 Why Separate Kernel Space and User Space?

The separation is important because user programs are not always trusted.

Benefits:

- Security: user programs cannot directly damage system resources
- Stability: one application crash does not usually crash the whole OS
- Isolation: processes cannot freely read or write each other's memory
- Controlled access: hardware and files are accessed through OS rules

Example:

If a browser crashes, the OS usually keeps running. This is because the browser runs in user space. If kernel code crashes, the entire system may crash.

5.4 Comparison Table

Feature	User Space	Kernel Space
Runs	Application programs	Kernel and core OS code
Privilege level	Low	High
Hardware access	Not direct	Direct
Memory access	Own process memory only	Full system memory access
Failure impact	Usually only the process crashes	Whole system may crash
Examples	Browser, editor, shell	Scheduler, memory manager, device drivers

6. System Calls

A system call is a controlled entry point through which a user program requests a service from the operating system kernel.

User programs cannot directly access many hardware and OS resources. Instead, they ask the kernel to perform operations on their behalf.

In simple terms:

A system call is the interface between user programs and the kernel.

6.1 Why Are System Calls Needed?

System calls are needed because normal programs run in user space with limited privileges.

Operations such as reading a file, creating a process, or sending data over a network require kernel help.

Examples of operations that need system calls:

- Create a process
- Terminate a process
- Open a file
- Read from a file
- Write to a file
- Allocate memory
- Communicate over a network
- Get current time
- Create a pipe

6.2 How a System Call Works

General flow:

1. A user program calls a library function, such as `printf`, `read`, or `open`.
2. The library function prepares the system call number and arguments.
3. The program executes a special instruction to switch from user mode to kernel mode.
4. The kernel checks the request.
5. The kernel performs the requested operation if allowed.
6. The kernel returns the result to the user program.
7. Execution continues in user mode.

Example:

When a C program calls:

```
read(fd, buffer, size);
```

The program is asking the OS to read data from a file descriptor into a buffer. The actual file access is done by the kernel.

6.3 Types of System Calls

Process Control

Used to create, terminate, and manage processes.

Examples:

- `fork()`
- `exec()`
- `wait()`
- `exit()`

File Management

Used to create, open, read, write, and delete files.

Examples:

- `open()`
- `read()`
- `write()`
- `close()`
- `unlink()`

Device Management

Used to access hardware devices.

Examples:

- Read from keyboard
- Write to display
- Control printer
- Access disk

Information Maintenance

Used to get or set system information.

Examples:

- Get current time
- Get process ID
- Get system information

Communication

Used for communication between processes.

Examples:

- Pipes
- Shared memory
- Message queues
- Sockets

6.4 System Call vs Function Call

Feature	Function Call	System Call
Purpose	Calls code inside the same program or library	Requests service from OS kernel
Mode switch	Usually no mode switch	Switches from user mode to kernel mode
Cost	Faster	Slower due to mode switch and checks
Privileges	User-level	Kernel-level service
Example	<code>sum(a, b)</code>	<code>read(fd, buf, n)</code>

Important interview point:

Every system call involves extra overhead compared to a normal function call because it requires switching from user mode to kernel mode.

7. Program vs Process

A program and a process are related, but they are not the same.

7.1 Program

A program is a passive file stored on disk.

It contains instructions written by a programmer and compiled or interpreted for execution.

Examples:

- `chrome.exe`
- `notepad.exe`
- `a.out`
- A Python script
- A Java `.class` or `.jar` file

A program is not executing by itself. It is just stored code.

7.2 Process

A process is an active instance of a program in execution.

When a program is loaded into memory and starts running, it becomes a process.

A process includes:

- Program code
- Program counter
- CPU registers
- Stack
- Heap
- Data section

- Open files
- Process ID
- Process state
- Scheduling information

In simple terms:

Program is passive. Process is active.

7.3 Example

Suppose `notepad.exe` is stored on disk.

- The file `notepad.exe` is a program.
- When you open Notepad, the OS creates a process.
- If you open Notepad three times, there is still one program file but three separate processes.

Each process has its own:

- Memory
- Process ID
- Stack
- Open files
- Execution state

7.4 Comparison Table

Feature	Program	Process
Nature	Passive	Active
Location	Stored on disk	Loaded in main memory
Execution	Not executing	Currently executing or waiting
Resources	Does not own CPU/memory actively	Owens resources such as memory and file descriptors
Lifespan	Exists until deleted	Exists from creation to termination
Example	<code>calculator.exe</code> file	Running calculator instance

8. Process Memory Layout

A process usually has several memory sections.

8.1 Text Section

Contains the executable program code.

This section is usually read-only to prevent accidental modification of instructions.

8.2 Data Section

Contains global and static variables.

It is often divided into:

- Initialized data: global variables with assigned values
- Uninitialized data: global variables without explicit values, often called BSS

8.3 Heap

Used for dynamic memory allocation.

Examples:

- `malloc` in C
- `new` in C++
- dynamically created objects in many languages

The heap usually grows upward in memory.

8.4 Stack

Used for function calls.

The stack stores:

- Local variables
- Function parameters
- Return addresses
- Saved registers

The stack usually grows downward in memory.

8.5 Process Control Block

The OS maintains information about each process in a data structure called the Process Control Block, or PCB.

PCB contains:

- Process ID
- Process state
- Program counter
- CPU registers
- CPU scheduling information
- Memory management information
- Accounting information
- I/O status information

The PCB is very important during context switching.

9. Process States

A process does not stay in one condition throughout its lifetime. It moves between different states.

The common five-state model includes:

- New
- Ready
- Running
- Waiting or Blocked
- Terminated

9.1 New State

A process is in the new state when it is being created.

The OS is setting up:

- Process ID
- PCB
- Memory allocation
- Initial resources

After creation, the process usually moves to the ready state.

9.2 Ready State

A process is in the ready state when it is ready to execute but waiting for CPU allocation.

It has all required resources except the CPU.

Example:

Many processes may be ready, but if the CPU is busy running another process, they must wait in the ready queue.

9.3 Running State

A process is in the running state when it is currently executing on the CPU.

In a single-core CPU, only one process can be running at a time. In a multi-core CPU, multiple processes can run simultaneously, one on each core.

9.4 Waiting or Blocked State

A process is in the waiting state when it cannot continue until some event occurs.

Common reasons:

- Waiting for I/O completion
- Waiting for user input
- Waiting for a file read
- Waiting for network data
- Waiting for a lock or semaphore

Example:

If a process requests data from disk, it may move to the waiting state until the disk operation completes.

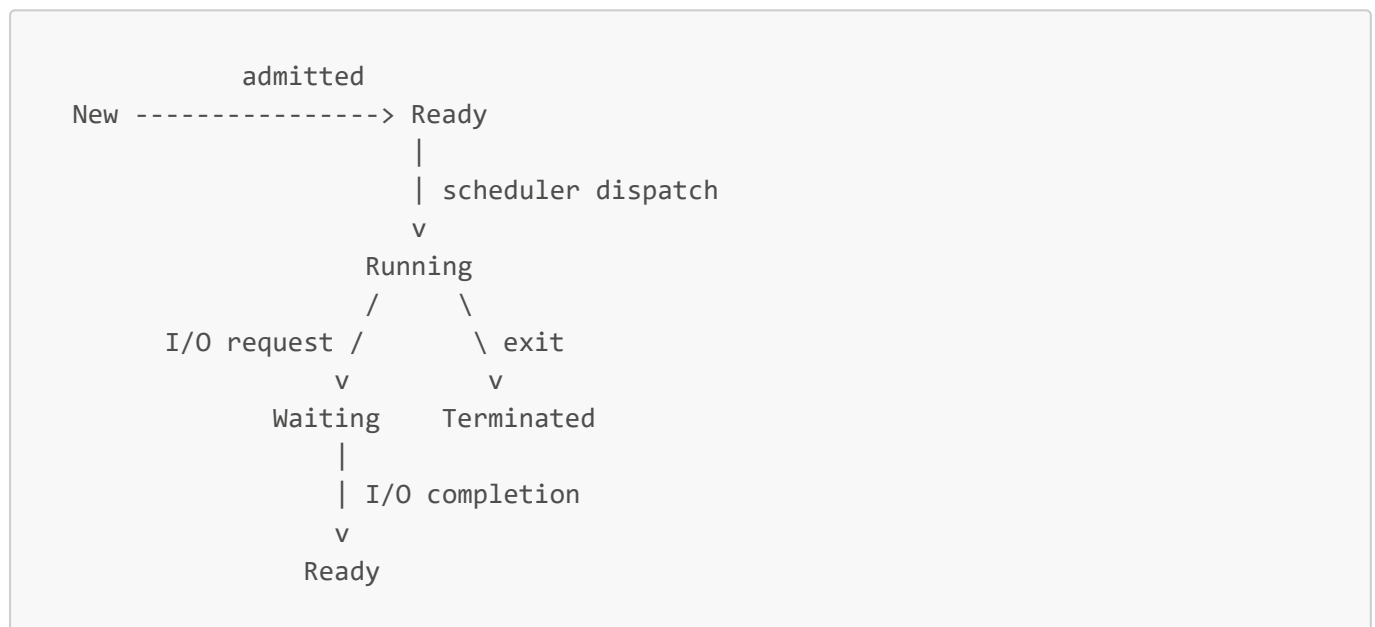
9.5 Terminated State

A process enters the terminated state after finishing execution or being killed.

The OS then releases its resources:

- Memory
- Open files
- PCB
- I/O resources

9.6 Process State Diagram



9.7 Important Transitions

New to Ready

The process has been created and admitted into the ready queue.

Ready to Running

The CPU scheduler selects the process for execution.

Running to Ready

The process is interrupted or its time quantum expires.

Example:

In Round Robin scheduling, if a process uses its time slice, it moves back to the ready queue.

Running to Waiting

The process requests I/O or waits for an event.

Example:

A process calls `read()` and waits for disk data.

Waiting to Ready

The event the process was waiting for has completed.

Example:

Disk read completes, so the process becomes ready again.

Running to Terminated

The process finishes or is forcefully killed.

10. Context Switching

Context switching is the process of saving the state of one process and loading the saved state of another process so that the CPU can switch from one process to another.

In simple terms:

Context switching allows the CPU to stop running one process and start or resume another process.

10.1 What Is Context?

The context of a process is the information needed to pause and later resume that process correctly.

It includes:

- Program counter
- CPU registers
- Stack pointer
- Process state
- Memory management information
- Open file information
- Scheduling information

Most of this information is stored in the process's PCB.

10.2 Why Context Switching Is Needed

Context switching is needed for multitasking.

It happens when:

- A process's time quantum expires

- A process performs I/O and becomes blocked
- A higher-priority process arrives
- An interrupt occurs
- The process voluntarily yields the CPU

Example:

You are listening to music while typing in a document. The CPU rapidly switches between the music player, editor, OS services, and other processes. This creates the illusion that they are running at the same time.

10.3 Steps in Context Switching

1. An interrupt, system call, or scheduler event occurs.
2. The OS stops the currently running process.
3. The OS saves the current process's context in its PCB.
4. The OS updates the process state, such as running to ready or waiting.
5. The scheduler selects another process from the ready queue.
6. The OS loads the selected process's saved context.
7. The selected process starts or resumes execution.

10.4 Context Switching Overhead

Context switching is necessary, but it is not free.

During a context switch, the CPU is not doing useful work for user programs. It is spending time saving and restoring states.

Overhead may include:

- Saving registers
- Restoring registers
- Updating PCBs
- Running the scheduler
- Switching memory address spaces
- Cache misses after switching
- Translation Lookaside Buffer, or TLB, effects

Too many context switches can reduce performance.

10.5 Context Switch vs Mode Switch

A mode switch changes the CPU from user mode to kernel mode or from kernel mode to user mode.

A context switch changes the currently running process or thread.

Feature	Mode Switch	Context Switch
Meaning	Switch between user mode and kernel mode	Switch from one process/thread to another

Feature	Mode Switch	Context Switch
Process changes?	Not necessarily	Yes, usually
Example	System call	Scheduler switches from Process A to Process B
Cost	Lower than full context switch	Usually higher

Important:

Every system call causes a mode switch, but not every system call causes a context switch.

Example:

If Process A calls `getpid()`, the CPU switches to kernel mode and returns to the same process. This is a mode switch, not necessarily a context switch.

If Process A calls `read()` and must wait for disk data, the OS may switch to Process B. That involves a context switch.

11. Interview-Focused Examples

Example 1: Opening a File

Suppose a program wants to read a file.

1. The program calls a library function like `fopen()` or `open()`.
2. The library eventually triggers a system call.
3. CPU switches from user mode to kernel mode.
4. Kernel checks permissions.
5. Kernel accesses the file system and disk driver.
6. If data is available, it returns data to the program.
7. If data is not immediately available, the process may go to waiting state.
8. Another process may be scheduled.

Concepts involved:

- User space
- Kernel space
- System call
- File management
- Process state transition
- Context switch

Example 2: Running a Program

Suppose you double-click a text editor.

1. The OS loads the program from disk into memory.

2. A new process is created.
3. The OS creates a PCB for the process.
4. The process enters the ready state.
5. The scheduler selects it for execution.
6. It enters the running state.
7. If it waits for user input, it may enter the waiting state.
8. When you close it, it enters the terminated state.

Concepts involved:

- Program vs process
 - Process creation
 - Process states
 - CPU scheduling
 - Resource management
-

12. Common Interview Questions and Answers

Q1. What is the role of an operating system?

An operating system acts as an interface between users, applications, and hardware. It manages resources such as CPU, memory, files, and I/O devices. It also provides security, process management, memory management, file management, and device management.

Q2. What is the difference between kernel space and user space?

User space is where normal applications run with limited privileges. Kernel space is where the OS kernel runs with full privileges. User programs cannot directly access hardware or kernel memory; they must use system calls.

Q3. What is a system call?

A system call is a mechanism by which a user program requests a service from the operating system kernel. Examples include reading a file, creating a process, writing to a file, and communicating over a network.

Q4. Is a system call the same as a function call?

No. A function call usually calls code within the same program or library and does not require a privilege change. A system call transfers control from user mode to kernel mode, so it is more expensive.

Q5. What is the difference between a program and a process?

A program is a passive file stored on disk. A process is an active instance of a program in execution. One program can have multiple running processes.

Q6. What are the main process states?

The main process states are new, ready, running, waiting or blocked, and terminated.

Q7. What is context switching?

Context switching is the process of saving the current process's state and loading another process's state so that the CPU can switch execution from one process to another.

Q8. Why is context switching expensive?

Context switching is expensive because the OS must save and restore CPU registers, update process control blocks, run the scheduler, switch address spaces, and may cause cache or TLB misses.

Q9. Can a process move from waiting directly to running?

Usually no. When the event it was waiting for completes, the process moves from waiting to ready. The scheduler later selects it to move from ready to running.

Q10. Does every system call cause a context switch?

No. Every system call generally causes a mode switch from user mode to kernel mode, but it does not always switch to another process. A context switch occurs only when the CPU changes from one process or thread to another.

13. Quick Revision Summary

- OS manages hardware and provides services to applications.
- Kernel is the core of the OS.
- User space runs normal applications with limited privileges.
- Kernel space runs privileged OS code.
- System calls allow user programs to request kernel services.
- A program is passive code stored on disk.
- A process is an active executing instance of a program.
- Process states describe the lifecycle of a process.
- Ready means waiting for CPU.
- Waiting means waiting for an event or I/O.
- Running means currently executing on CPU.
- Context switching enables multitasking.
- Context switching has overhead, so too many switches reduce performance.
- Mode switch and context switch are different.

14. One-Line Placement Answers

Use these for fast oral revision:

- OS: Software that manages hardware resources and provides services to programs.
- Kernel: Core part of the OS that directly manages hardware and system resources.
- User space: Memory area where normal applications run with limited privileges.
- Kernel space: Memory area where privileged OS code runs.
- System call: Interface through which a user program requests kernel services.
- Program: Passive executable file stored on disk.
- Process: Program in execution.

- Ready state: Process is ready to run but waiting for CPU.
- Running state: Process is currently using the CPU.
- Waiting state: Process is waiting for I/O or an event.
- Context switch: Saving one process's state and loading another process's state.